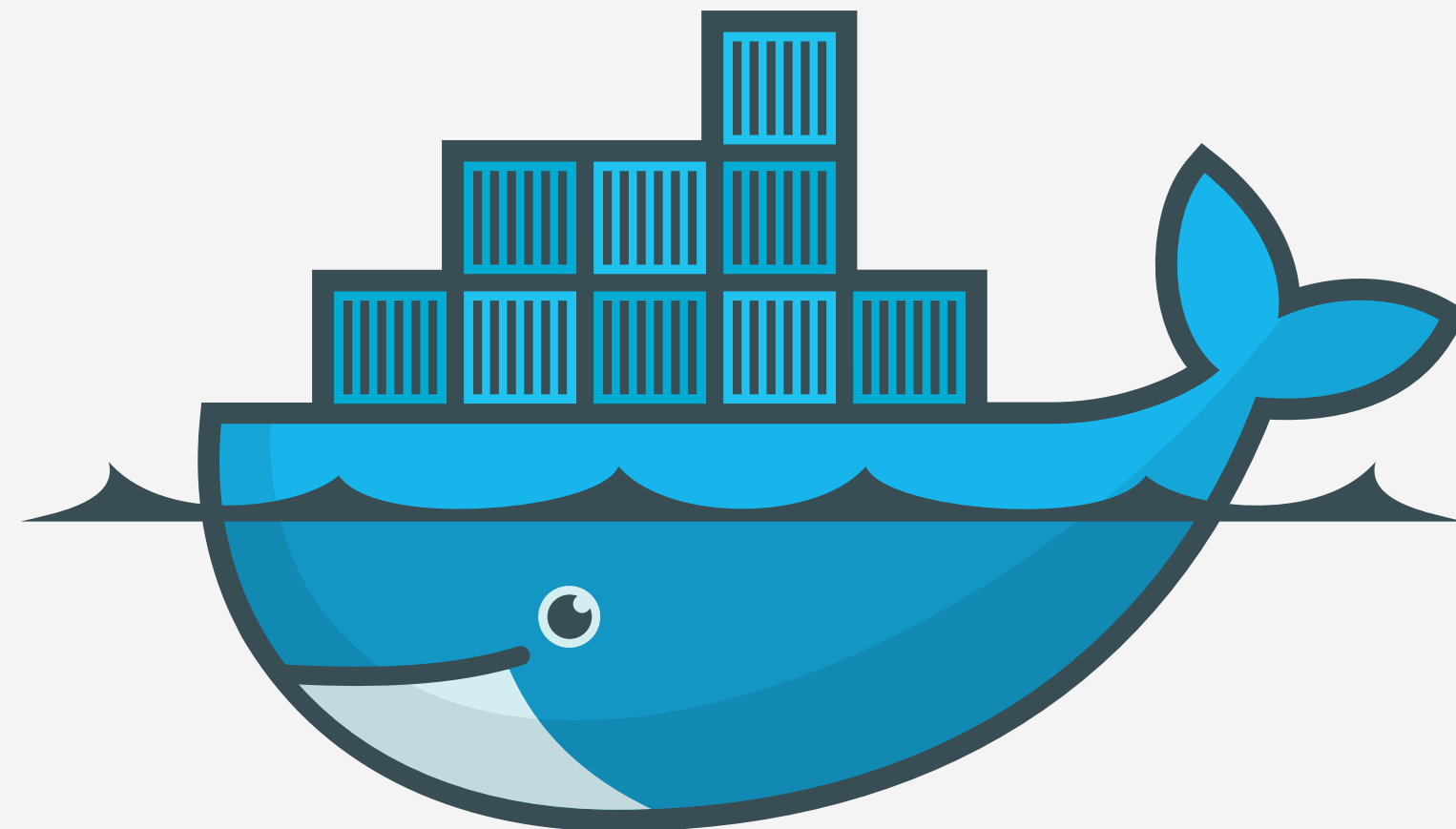




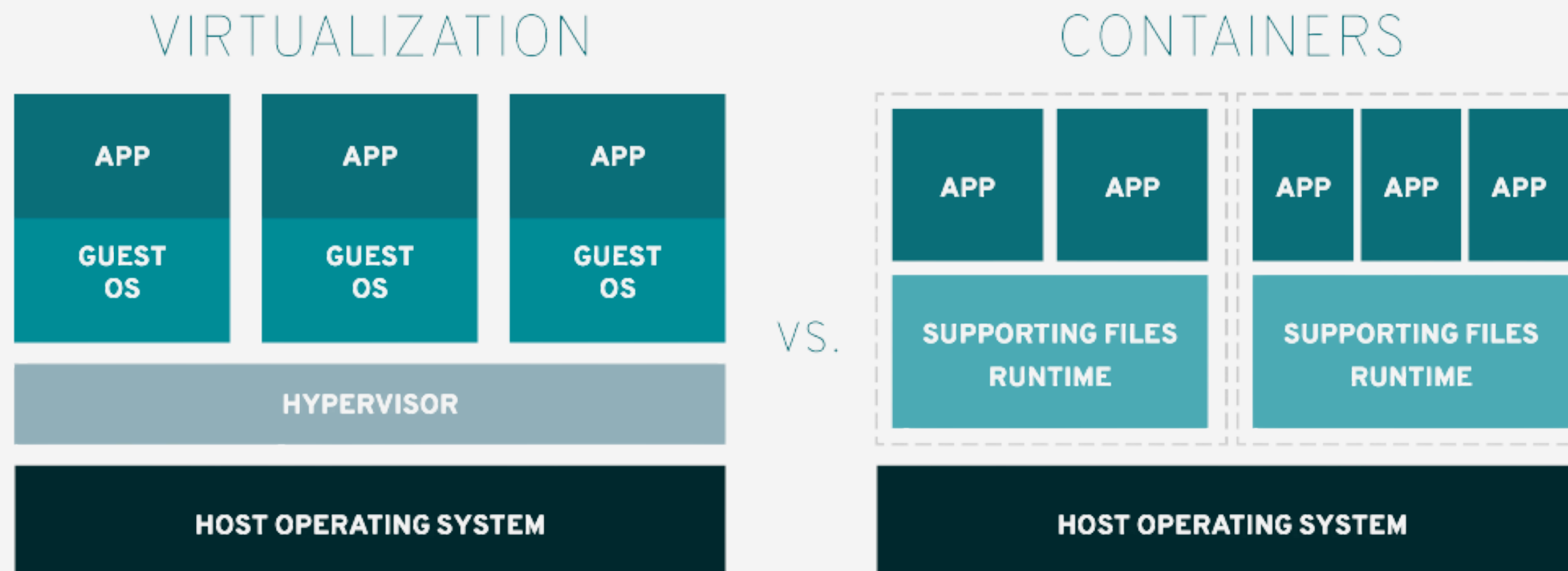
Факултет по Математика и Информатика
ПУ "Паисий Хилендарски"

ВИРТУАЛИЗАЦИЯ КОНТЕЙНЕРИЗАЦИЯ DOCKER

Витали Атиас



Виртуализация и контейнеризация



Какво представява виртуализацията?

- Технология, която позволява няколко операционни системи да работят на една физическа машина.
- Използва виртуални машини (VM) – всяка със своя ОС, ресурси и приложения.
- Осигурява изолация, гъвкавост и по-добро използване на хардуера.

Какво представява виртуализацията?

Видове хипервайзори

Тип 1 (Bare-Metal)

- Работи директно върху хардуера
- По-висока производителност и сигурност
- Примери: VMware ESXi, Microsoft Hyper-V, Xen

Тип 2 (Хостван)

- Работи върху съществуваща ОС
- По-лесен за настройка, но по-бавен
- Примери: Oracle VirtualBox, VMware Workstation

Какво представява виртуализацията?

Предимства на виртуализацията

- Изолиране – всяка виртуална машина е независима; проблем в една не влияе на останалите.
- Управление на ресурсите – възможност за точно разпределение на процесор, памет и място за съхранение според нуждите.
- Разнообразие на операционни системи – позволява работа на различни ОС (например Windows и Linux) върху един и същ хардуер.

Какво представява виртуализацията?

Случаи на използване на виртуализацията

- Работа с наследени приложения – позволява старите приложения да функционират в необходимата операционна среда.
- Консолидация на сървъри – няколко сървъра могат да работят на една физическа машина, което намалява разходите и опростява управлението.
- Тестване и разработка – създаване на изолирани среди за тестване и разработване на приложения с различни конфигурации на ОС.

Какво представява контейнери- зацията?

- Контейнеризацията е олекотена алтернатива на виртуализацията.
- Приложенията и техните зависимости се пакетират в контейнери, които могат да се изпълняват последователно и предсказуемо навсякъде.
- За разлика от виртуалните машини, контейнерите споделят ядрото на хост операционната система, което ги прави:
 - по-ефективни по отношение на ресурсите,
 - по-бързи за стартиране,
 - и по-лесни за пренасяне.

Какво представява контейнери- зацията?

- Контейнерите се управляват от контейнерна среда за изпълнение (runtime), като: Docker, containerd, Podman или други.
- Те се изпълняват върху хост операционната система, използвайки:
 - Namespaces – за изолация на процеси, мрежи и файлови системи;
 - cgroups – за управление на ресурси.
- Всеки контейнер има:
 - собствена файлова система,
 - собствена мрежа,
 - собствено пространство за процеси,
 - докато споделя ядрото на хост системата.

Предимства на контейнери- зацията

1. Лекота

- Контейнерите са много по-малки от виртуалните машини.
- Споделят ядрото на хост системата → по-бързо стартиране и по-ниска консумация на ресурси.

2. Преносимост

- Контейнерите могат да се преместват лесно между среди – разработка, тестване и продукция.
- Включват всички зависимости, необходими за изпълнение на приложението.

3. Микросървисна архитектура

- Подходящи за микроуслуги – всяка услуга в отделен контейнер.
- Позволяват независимо разработване, обновяване и мащабиране на отделните компоненти.

Недостатъци и предизвикателства

1. Сложност при управление

- Голяма бройка контейнери изисква инструменти за оркестрация, като Kubernetes, което добавя сложност.

2. Сигурност

- Тъй като контейнерите споделят ядрото на операционната система, неправилна конфигурация може да изложи системата на риск.

3. Постоянно съхранение и мрежи

- Управлението на персистентни данни и мрежова свързаност между контейнери може да бъде предизвикателно.

Случаи на използване на контейнеризацията

1. Разгръщане на микросървиси

- Позволява приложенията да се разделят на по-малки, независими услуги.
- Всяка услуга може да бъде внедрявана и мащабирана отделно, без да се засягат останалите.

2. CI/CD (Непрекъснатата интеграция и доставка)

- Контейнерите улесняват автоматизацията на процесите по изграждане, тестване и внедряване.
- Те са основен компонент в DevOps pipelines, осигурявайки бързи и стабилни версии.

3. Изолиране на приложенията

- Контейнерите изолират приложенията и техните зависимости.
- Промените в един контейнер не влияят върху останалите, което повишава стабилността и сигурността.

Кога да използваме виртуализация и контейнеризация?

Критерий	Виртуализация	Контейнеризация
Основна идея	Стартира няколко виртуални машини с отделни операционни системи.	Стартира няколко изолирани контейнера, споделящи една ОС.
Подходяща, когато...	Трябва да използвате няколко ОС на една машина; Приложенията имат различни зависимости; Изискват се специфични версии на ОС; Нужна е по-силна изолация между натоварвания.	Използвате микроуслуги с отделни зависимости.- Търсите мащабируемост и преносимост.; Искате бързо стартиране и ниско ресурсно натоварване.;Работите по DevOps модел с CI/CD.
Примери за употреба	Хостинг на различни уеб сървъри или стари приложения.	Разработка и внедряване на съвременни приложения с микроуслуги.
Изолация	Пълна – всяка VM има собствена ОС.	Лека – контейнерите споделят ядрото на ОС.
Скорост и ресурси	По-бавно стартиране и по-високо използване на ресурси.	Много бързо стартиране и оптимално използване на ресурси.

Комбиниране на виртуализация и контейнеризация

Сценарий	Ползи
Контейнери върху виртуални машини	- Осигурява силна изолация между различни приложения. - Позволява стар и нов софтуер да работят на една инфраструктура. - Контейнери дават бързо стартиране, лесно мащабиране и преносимост.
Мулти-облачна инфраструктура	- Виртуализацията осигурява съвместимост между различни облачни платформи. - Контейнерите улесняват миграция и внедряване на приложения.
DevOps и CI/CD среди	- Контейнерите ускоряват тестове и внедряване. - Виртуалните машини изолират средите и предотвратяват конфликти между зависимости.

Контейнеризация

История и еволюция

Историята на контейнерите започва много преди Docker и корените му могат да се проследят до ранните опити за изолация на процеси в Unix системи.

1979 - Unix V7 и chroot

- Въведен от Bill Joy по време на разработката на BSD Unix
- chroot променя root директорията за процес и неговите подпроцеси
- Първата форма на filesystem изолация
- Използван главно за тестване и сигурност

Ограничения на chroot:

- Лесно се заобикаля с root привилегии
- Няма изолация на процеси, мрежа или ресурси
- Само filesystem изолация

История и еволюция

1998-2000 - FreeBSD Jails

- По-напреднала изолация от chroot
- Виртуализация на операционната система
- Всяка jail има своя hostname, IP адрес
- Изолация на filesystem, процеси, потребители
- Използван за shared hosting providers

2001 - Linux VServer

- Виртуализация на Linux kernel
- Изолация на процеси и мрежа

2004 - Solaris Containers (Zones) - Oracle

- Комбинация от resource management и виртуализация
- Много стабилна и производителна система

История и еволюция

2005 - OpenVZ

- Open source container-based виртуализация за Linux
- Kernel патчове за изолация
- Популярен при hosting providers
- Shared kernel между containers

2006 - Process Containers (по-късно cgroups)

- Разработен от Google инженери (Paul Menage, Rohit Seth)
- Resource limitation и prioritization
- Вграден в Linux kernel 2.6.24 (2008)
- Фундаментална технология за модерните контейнери

История и еволюция

2008 - LXC (Linux Containers)

- Първата пълна имплементация на Linux container manager
- Комбинираща cgroups и kernel namespaces
- User-space tools за управление
- По-близо до модерните контейнери

2013 - Docker дебютира

- Solomon Hykes, dotCloud (сега Docker Inc.)
- Първоначално built върху LXC
- По-късно разработен libcontainer (сега runc)
- Революционизира начина, по който се използват контейнери
- Въведе image format и layered filesystem
- Docker Hub за споделяне на images

История и еволюция

Защо Docker промени всичко:

- Лесна употреба (developer-friendly)
- Portable images
- Version control на environments
- Dockerfile за reproducible builds
- Огромна ecosystem и community
- Industry standardization

2014 - Kubernetes от Google

- Open source на вътрешната система Borg
- Container orchestration platform
- Станал индустриален стандарт
- Cloud Native Computing Foundation (CNCF)

2015+

- Масово приемане в индустрията
- Cloud providers обръщат внимание
- Kubernetes става de facto orchestrator
- Serverless containers
- Container standards (OCI)

Класическият проблем

"It works on my machine"

Среда в software lifecycle

Development → Testing → Staging → Production

Всяка среда може да има:

- Различни OS versions
- Различни dependencies
- Различни configurations
- Различни runtime versions

Традиционни решения:

1. Различни сървъри - Скъпо, неефективно
2. Virtual environments - Частично решение, не изолира всичко
3. Virtual machines - Тежки, бавен startup
4. Контейнери - Оптималното решение

Docker

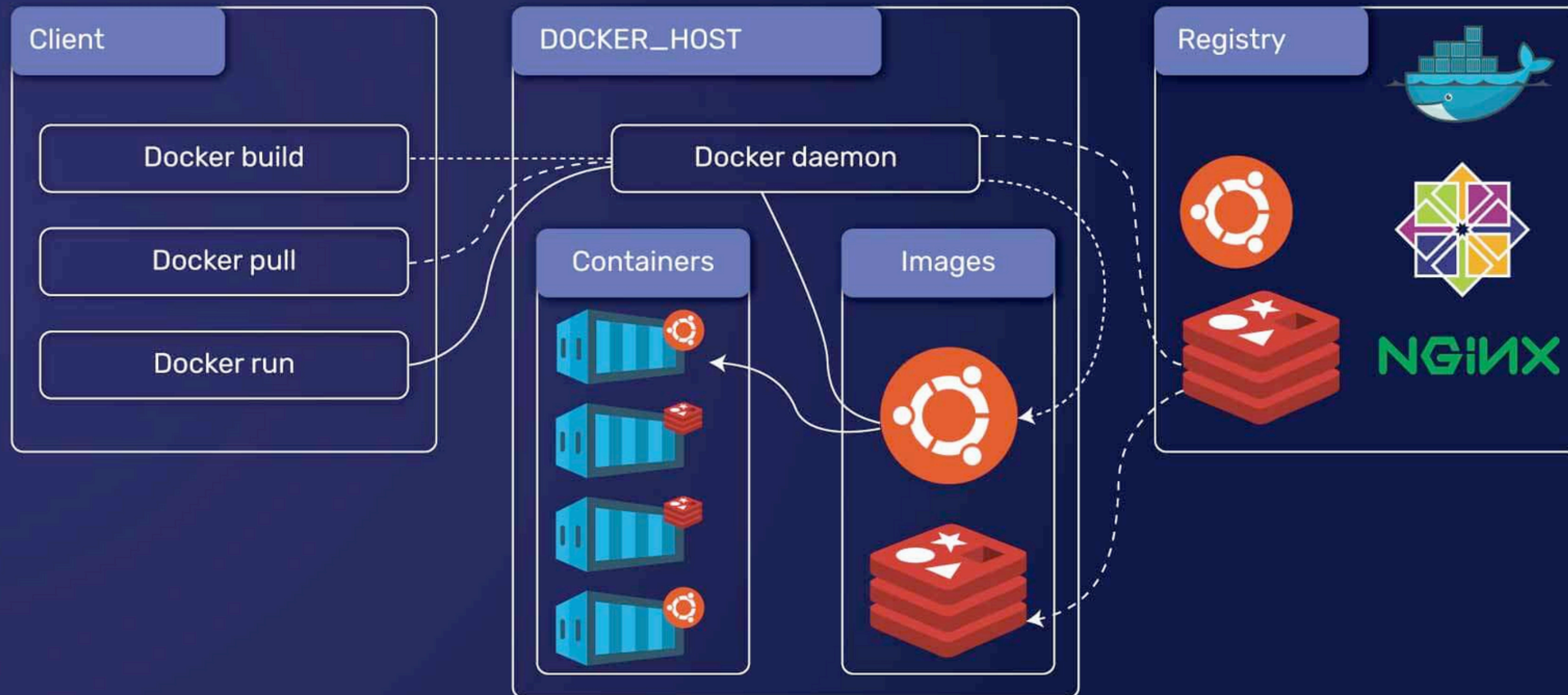
```
#docker run hello-world
```

Какво е Docker

- Платформа за създаване, разпространение и стартиране на контейнери.
- Контейнерите осигуряват еднаква работа на приложенията навсякъде – локално, сървъри или облак.
- Позволява бързо внедряване и по-лесно управление на зависимости.



Docker Container Architecture



Основни концепции

Image

- Read-only шаблон, който съдържа приложение и всички необходими зависимости, библиотеки и конфигурации.
- Използва се като основа за стартиране на контейнер.
- Може да се базира на друг image (например Ubuntu, Alpine) и да се разширява чрез Dockerfile.

Container

- Стартирана инстанция на image, която е изолирана от хост системата и други контейнери.
- Контейнерът съдържа собствената си файловата система, процеси и мрежа, но използва ядро на хоста (Linux kernel или Windows kernel).
- Лесен за стартиране, спиране, клониране и мащабиране.

Основни концепции

Dockerfile

- Текстов файл с инструкции за изграждане на image.
- Позволява автоматизация на процеса на билд, като дефинира базовия image, инсталира необходимия софтуер, копира файлове и задава стартова команда.
- Всеки ред обикновено създава слой, което позволява ефективно кеширане при повторно билдване.

Docker Engine

- Софтуерен слой, който управлява build, run, stop и управление на контейнерите.
- Състои се от Docker daemon (основният процес) и CLI инструмент, чрез който се управляват контейнерите.
- Осигурява комуникация между контейнерите, images и хост системата.

Основни концепции

Registry (Docker Hub)

- Място за съхранение, споделяне и теглене на images.
- Може да съдържа официални images (например Ubuntu, Python) или потребителски създадени images.
- Позволява лесно разпространение на приложения между екипи и среди.

Bugove Docker images

- Full OS – пълна операционна система с всички инструменти.
- Slim / Minimal – малък image с минимални файлове и зависимости.
- Scratch – празен image, започваш от нулата.
- Application – конкретно приложение с всички нужни зависимости.
- Builder / Runtime – за компилиране на приложения или предоставяне на runtime библиотеки.

Предимства на Docker

- Преносимост – контейнерът работи еднакво навсякъде.
- Бързо стартиране и малко използване на ресурси.
- Изолация – контейнери са отделени един от друг и от хост системата.
- Подходящ за мащабиране и интеграция в CI/CD конвейри.
- Улеснява управление на зависимости и микроуслуги.

Стандарти и съвместимост

- OCI (Open Container Initiative) – стандарт за контейнерни images и runtime.
- Docker images днес са OCI-съвместими, което осигурява преносимост между платформи.
- Linux контейнери използват Linux kernel; Windows контейнери – Windows kernel.
- На Windows и macOS Linux контейнерите се стартират чрез виртуална Linux среда (WSL2 или VM).

Dockerfile – съдържание

- FROM – базов image (OS или минимален образ).
- RUN – команди за инсталиране на софтуер или настройка на системата.
- COPY / ADD – добавяне на файлове и директории в контейнера.
- WORKDIR – задава работна директория за команди и стартиране.
- CMD / ENTRYPOINT – команда по подразбиране при стартиране на контейнера.
- ENV / ARG – задаване на променливи на средата или build-time аргументи.
- EXPOSE, USER, VOLUME, LABEL, HEALTHCHECK – допълнителна конфигурация, портове и проверки.

Оптимизация на слоевете

- Групирай свързани команди в един RUN ред с &&.
- Почистване на кеш и временни файлове в същия слой.
- Минимизирай броя на слоевете и COPY/ADD инструкции.
- Multi-stage builds – отделя build инструменти от финалния image за по-малък размер.

```
# 1. Базов image
FROM python:3.12-slim

# 2. Аргументи за build
ARG APP_ENV=production
ENV APP_ENV=${APP_ENV}

# 3. Работна директория
WORKDIR /app

# 4. Копиране на зависимостите първо за кеширане
COPY requirements.txt /app/

# 5. Инсталиране на зависимости
RUN pip install --no-cache-dir -r requirements.txt

# 6. Копиране на приложението
COPY . /app/

# 7. Порт, който ще слуша контейнера
EXPOSE 5000

# 8. Потребител за стартиране
RUN useradd -m appuser
USER appuser

# 9. Healthcheck
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s \
  CMD curl -f http://localhost:5000/ || exit 1

# 10. Стартова команда
CMD ["python", "app.py"]
```

```
# Stage 1: Build
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

# Stage 2: Final image
FROM alpine:3.18
COPY --from=builder /app/myapp /usr/local/bin/
CMD ["myapp"]
```

Упражнение

```
# Проверка на Docker версия  
docker version
```

```
# System информация  
docker system info
```

```
# Помощ  
docker --help  
docker run --help
```

```
# Примерна програма  
docker run hello-world
```

```
# Изтегляне на image  
docker pull ubuntu:22.04  
docker pull nginx:latest
```

```
# Листване на images  
docker images  
docker image ls
```

```
# История на image  
docker image history nginx:latest
```

```
# Изтриване на image  
docker rmi nginx:latest
```

```
# Run container (foreground)
docker run ubuntu:22.04 echo "Hello World"
```

```
# Run container (background/detached)
docker run -d nginx:latest
```

```
# Run с име
docker run -d --name mynginx nginx:latest
```

```
# Run интерактивен
docker run -it ubuntu:22.04 bash
```

```
# Run с port mapping
docker run -d -p 8080:80 nginx:latest
# Host port 8080 -> Container port 80
```

```
# Спиране на container  
docker stop mynginx
```

```
# Стартиране на stopped container  
docker start mynginx
```

```
# Restart  
docker restart mynginx
```

```
# Pause/Unpause  
docker pause mynginx  
docker unpause mynginx
```

```
# Kill (SIGKILL)
```

```
docker kill mynginx
```

```
# Remove container
```

```
docker rm mynginx
```

```
docker rm -f mynginx # Force remove running container
```

```
# Run Ubuntu container интерактивно
docker run -it ubuntu:22.04 bash

# Вътре в container-a, execute:
cat /etc/os-release
# Вижте Ubuntu information

ls /
# Filesystem на container-a

apt-get update
apt-get install -y curl

curl --version

# Check процеси
ps aux
# Забележете: Само процеси в container-a

# Check network
ip addr
hostname

# Exit
exit

# Container е stopped след exit
docker ps -a | grep ubuntu
```

```
# Run Nginx web server
docker run -d -p 8080:80 --name myweb nginx:latest

# Verify running
docker ps

# Test в browser или с curl
curl http://localhost:8080
# Трябва да видите Nginx welcome page

# View logs
docker logs myweb

# Exec в container
docker exec -it myweb bash

# Вътре в container
ls /usr/share/nginx/html
cat /usr/share/nginx/html/index.html
exit

# Stop и remove
docker stop myweb
docker rm myweb

# Cleanup image
docker rmi nginx:latest
```

```
# Memory limit
docker run -d --name memlimit --memory="100m" alpine sleep 3600

# Verify limit
docker inspect memlimit | grep Memory

# CPU limit
docker run -d --name cpulimit --cpus="0.5" alpine sleep 3600

# Monitor resources
docker stats memlimit cpulimit

# Cleanup
docker rm -f memlimit cpulimit
```

За самостоятелна работа

```
#Създайте директория  
my-node-app/
```

```
#Създайте два празни файла  
app.js  
package.json
```

1

package.json

```
{  
  "name": "my-node-app",  
  "version": "1.0.0",  
  "main": "app.js",  
  "scripts": {  
    "start": "node app.js"  
  },  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

2

```
# Базов образ
FROM node:20-alpine

# Работна директория
WORKDIR /app

# Копиране на зависимостите
COPY package*.json ./

# Инсталиране на зависимости
RUN npm install

# Копиране на приложението
COPY . .

# Порт, който контейнерът ще слуша
EXPOSE 3000

# Стартова команда
CMD ["npm", "start"]
```

```
const express = require('express');
const app = express();
const port = process.env.PORT ||
3000;

app.get('/', (req, res) => {
  res.send('Hello from Docker
Node.js app!');
});

app.listen(port, () => {
  console.log(`App running on port
${port}`);
});
```

```
#Създайте image
docker build -t my-node-app .

#Стартирайте container
docker run -p 3000:3000 my-node-app
```

5

